

Implémentation de Wasserstein GAN et Energy-based Model pour Generative Adversarial Networks

Alexis Michel & Tomil Shi - Zhou Yang-Bo

Projet GenAI : M2 IASD 2025-2026



Contents

1	Introduction	2
2	Wasserstein GAN with Gradient Penalty	2
2.1	Présentation de la méthode	2
2.2	Points clés de l'algorithme	2
2.3	Implémentations et Adaptations	3
3	Your GAN is Secretly an Energy-based Model and You Should Use Discriminator Driven Latent Sampling	4
3.1	Présentation de la méthode	4
3.2	Points clés de l'algorithme	4
3.3	Implémentation et Adaptations	5
4	Comparaison et Résultats expérimentaux	5
5	Limites de l'implémentation globale	6

1 Introduction

Le projet vise à implémenter une ou plusieurs méthodes afin d'améliorer les Generative Adversarial Networks (GANs) dans le but d'obtenir une meilleure génération d'images. Ce rapport met en avant deux améliorations complémentaires, issues de travaux de recherche récents.

La première se base sur les travaux proposés dans Improved Training of Wasserstein GANs (Wasserstein GAN with Gradient Penalty), où nous modifions l'entraînement. La seconde méthode, issue du papier Your GAN is Secretly an Energy-based Model and You Should Use Discriminator Driven Latent Sampling, ne modifie pas l'entraînement du modèle mais propose une nouvelle façon de générer des images de meilleure qualité en exploitant le discriminateur comme modèle à base d'énergie.

Ces deux méthodes sont implémentées en Python avec PyTorch et testées sur le dataset MNIST. Elles sont ensuite comparées aux techniques de Soft Truncation et Hard Truncation. Afin d'évaluer et de comparer ces approches, nous utilisons les métriques FID, Précision et Recall.

2 Wasserstein GAN with Gradient Penalty

2.1 Présentation de la méthode

Les GANs standard ont un problème fondamental au niveau de leur fonction de perte, qui repose sur la divergence de Jensen-Shannon entre la distribution réelle p_r et la distribution générée p_g . Cette divergence sature lorsque les supports de p_r et p_g sont disjoints, ce qui est quasi-systématique en grande dimension car ces distributions reposent sur des variétés de faible dimension. Dans ce cas, la divergence de Jensen-Shannon est constante et égale à $\log 2$, rendant le gradient du générateur nul et inexploitable.

WGAN propose de remplacer la divergence de Jensen-Shannon par la **Earth-Mover (EM) distance**, définie comme le coût minimal de transport pour transformer p_g en p_r :

$$W(p_r, p_g) = \inf_{\gamma \in \Pi(p_r, p_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|] \quad (1)$$

où $\Pi(p_r, p_g)$ désigne l'ensemble de tous les couplages de p_r et p_g . Cependant, calculer cette distance directement est computationnellement trop coûteux. WGAN fait le choix de passer par sa **forme duale de Kantorovich-Rubinstein** :

$$W(p_r, p_g) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x \sim p_r} [f(x)] - \mathbb{E}_{x \sim p_g} [f(x)] \quad (2)$$

où la borne supérieure est prise sur l'ensemble des fonctions 1-Lipschitz. Cette distance est continue et différentiable presque partout, fournissant un gradient stable et exploitable tout au long de l'entraînement, là où la divergence de Jensen-Shannon échoue lorsque les supports sont disjoints.

2.2 Points clés de l'algorithme

Pour approximer la forme duale, le critic doit être une fonction 1-Lipschitz. Le papier original WGAN impose cette contrainte par **weight clipping** : les poids du critic sont tronqués dans un intervalle $[-c, c]$ après chaque mise à jour. WGAN-GP identifie plusieurs problèmes liés à cette approche : elle conduit à des difficultés d'optimisation et produit une surface de valeurs pathologique pour le critic. En particulier, le weight clipping pousse les poids vers les deux valeurs extrêmes de l'intervalle $[-c, c]$, ce qui limite la capacité du critic à capturer les moments d'ordre supérieur de la distribution. De plus, les normes de gradient ont tendance à exploser ou à disparaître lors de l'entraînement de critics profonds avec cette contrainte.

Le papier WGAN-GP propose à la place d'imposer la contrainte de Lipschitz via un **gradient penalty** directement intégré dans la fonction de perte. L'idée est d'encourager le critic à avoir une norme de gradient égale à 1 en tout point. Cette pénalité est calculée sur des points interpolés $\hat{x}$ entre données réelles et générées :

$$\hat{x} = \alpha x_{real} + (1 - \alpha)x_{fake}, \quad \alpha \sim \mathcal{U}[0, 1] \quad (3)$$

$$GP = \mathbb{E}_{\hat{x}} \left[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2 \right] \quad (4)$$

Le choix d'interpoler entre x_{real} et x_{fake} définit une distribution $p_{\hat{x}}$ sur laquelle le gradient penalty est calculé, ce qui permet d'imposer la contrainte de Lipschitz de manière douce et différentiable, contrairement au weight clipping qui l'impose de manière dure.

La fonction de perte complète du critic devient alors :

$$\mathcal{L}_D = \mathbb{E}_{x \sim p_g} [D(x)] - \mathbb{E}_{x \sim p_r} [D(x)] + \lambda \cdot GP \quad (5)$$

Et celle du générateur :

$$\mathcal{L}_G = -\mathbb{E}_{x \sim p_g} [D(x)] \quad (6)$$

où $\lambda = 10$ est le coefficient de régularisation donné par le papier. Notons que le critic est entraîné n_{critic} fois par mise à jour du générateur. Cela est nécessaire car le critic doit approximer correctement la distance de Wasserstein avant chaque step du générateur. Un critic sous-entraîné fournirait un signal de gradient inexact et dégraderait l'entraînement.

2.3 Implémentations et Adaptations

L'implémentation de WGAN-GP s'articule autour de trois éléments principaux dans notre code : le calcul du gradient penalty, la fonction de perte du critic, et la boucle d'entraînement.

La fonction `compute_gradient_penalty` dans `utils.py` implémente le calcul du GP en interpolant entre données réelles et générées, puis en calculant la norme du gradient du critic sur ces points :

```
alpha = torch.rand(batch_size, 1).expand_as(real_samples)
interpolates = (alpha * real + (1 - alpha) * fake).requires_grad_(True)
gradients = autograd.grad(D(interpolates), interpolates, ...)[0]
GP = ((gradients.norm(2, dim=1) - 1)**2).mean()
```

Les fonctions de perte du critic et du générateur sont implémentées dans `D_train` et `G_train` :

```
# Critic
d_loss = fake_validity.mean() - real_validity.mean() + lambda_gp * GP
```

```
# Generateur
g_loss = -fake_validity.mean()
```

Plusieurs adaptations ont été nécessaires par rapport au papier original.

Le papier WGAN recommande RMSProp, mais dans celui de WGAN-GP, une comparaison est faite entre Adam et RMSProp, et Adam est plus performant. Nous avons donc opté pour Adam et avons testé plusieurs valeurs de β_1 et β_2 , les valeurs $\beta_1 = 0.0$ et $\beta_2 = 0.9$ s'étant révélées les plus concluantes.

Le papier recommande $n_{critic} = 5$. Après plusieurs expériences, nous avons fixé cette valeur à 3. Une valeur de 5 rendait le critic trop dominant tandis qu'une valeur de 2 favorisait trop le générateur.

Un LR scheduling (StepLR, $\gamma = 0.5$ tous les 20 epochs) a été ajouté, réduisant le learning rate par un facteur 2 tous les 20 epochs. Cela s'est révélé essentiel pour éviter la divergence progressive de la G loss observée sans ce mécanisme.

3 Your GAN is Secretly an Energy-based Model and You Should Use Discriminator Driven Latent Sampling

3.1 Présentation de la méthode

Le papier propose d'améliorer la qualité de génération en exploitant conjointement le générateur et le discriminateur à travers des modèles à base d'énergie (EBM).

L'idée centrale repose sur l'observation suivante : l'entraînement d'un GAN ne converge généralement pas vers le générateur optimal, si bien que p_d et p_g ne coïncident pas parfaitement. Cependant, le discriminateur fournit une estimation quantitative de ce mismatch. En supposant le discriminateur proche de l'optimalité, on a :

$$D(x) \approx \frac{p_d(x)}{p_d(x) + p_g(x)} \quad (7)$$

En notant $d(x)$ le logit de $D(x)$, on peut montrer que $e^{d(x)} \approx p_d/p_g$, et donc :

$$p_d(x) \approx p_g(x)e^{d(x)} \quad (8)$$

Cette relation motive la définition du modèle à base d'énergie $p_d^* = p_g(x)e^{d(x)}/Z_0$, où Z_0 est une constante de normalisation. Le papier montre que lorsque $D = D^*$ est le discriminateur optimal, alors $p_d^* = p_d$.

Cependant, échantillonner directement depuis p_d^* dans l'espace pixel est impraticable en raison de la haute dimensionnalité et du long temps de mélange. Le papier propose alors de reformuler ce modèle dans l'espace latent via la fonction d'énergie, qui pour un prior gaussien standard $p_0(z) = \mathcal{N}(0, I)$ s'écrit :

$$E(z) = \frac{1}{2}\|z\|^2 - d(G(z)) \quad (9)$$

Si l'on échantillonne $z \sim p_t(z) \propto e^{-E(z)}$ et que l'on génère $x = G(z)$, alors $x \sim p_d^*$.

3.2 Points clés de l'algorithme

Pour échantillonner depuis $p_t(z) \propto e^{-E(z)}$, le papier utilise la **dynamique de Langevin**, un algorithme de Monte Carlo par Chaînes de Markov (MCMC) dont la mise à jour est :

$$z_{i+1} = z_i - \frac{\epsilon}{2}\nabla_z E(z_i) + \sqrt{\epsilon} \cdot \eta, \quad \eta \sim \mathcal{N}(0, I) \quad (10)$$

où ϵ est le pas et η un bruit gaussien. En développant le gradient de $E(z) = \frac{1}{2}\|z\|^2 - d(G(z))$:

$$\nabla_z E(z) = z - \nabla_z d(G(z)) \quad (11)$$

DDLS exploite ainsi l'information contenue dans le discriminateur pour corriger les biais et erreurs du générateur. Le papier montre que cette dynamique converge vers la distribution cible $p_t(z)$ lorsque $\epsilon \rightarrow 0$ et que le nombre d'itérations tend vers l'infini. En pratique, on utilise un ϵ petit mais fini, ce qui introduit un biais résiduel.

Un avantage majeur de DDLS est son efficacité d'échantillonnage. Le papier souligne que migrer l'échantillonnage de Langevin dans l'espace latent plutôt que dans l'espace image de haute dimension permet d'obtenir de meilleures performances en un nombre bien plus faible d'itérations, comparé à des méthodes comme MH-GAN qui nécessitent plus de 600 itérations MCMC pour atteindre des performances compétitives.

3.3 Implémentation et Adaptations

L’implémentation de DDLS dans `generate.py` suit directement l’Algorithme 1 du papier. Pour chaque batch, on initialise $z \sim \mathcal{N}(0, I)$ puis on effectue N étapes de dynamique de Langevin :

Algorithm 1 Discriminator Driven Latent Sampling

```

1: Initialiser  $z \sim \mathcal{N}(0, I)$  de dimension 100
2: for  $i = 0$  à  $N - 1$  do
3:    $\epsilon_i \leftarrow \epsilon_{min} + (\epsilon_0 - \epsilon_{min}) \times 0.5 \times (1 + \cos(\pi i / N))$ 
4:    $x \leftarrow G(z)$ 
5:    $E(z) \leftarrow \frac{1}{2} \|z\|^2 - \text{Sigmoid}(D(x))$ 
6:    $\nabla_z \leftarrow \nabla_z E(z)$ 
7:    $\eta \sim \mathcal{N}(0, I)$ 
8:    $z \leftarrow z - \frac{\epsilon_i}{2} \nabla_z + \sqrt{\epsilon_i} \cdot \eta$ 
9: end for
10: return  $x = G(z)$ 

```

Une fois les N étapes effectuées, on génère l’image finale avec $x = G(z)$.

Plusieurs adaptations ont été nécessaires.

Le papier formule DDLS pour un GAN standard où $d(x)$ est le logit du discriminateur, c’est-à-dire $D(x) = \sigma(d(x))$. Dans notre cas, le critic WGAN-GP produit un logit non borné sans sigmoid. Or la fonction d’énergie $E(z) = \frac{1}{2} \|z\|^2 - d(G(z))$ suppose que $d(G(z))$ est bien calibré. Nous avons donc appliqué un sigmoid sur la sortie du critic uniquement lors de la génération, ce qui re-borne la sortie dans $[0, 1]$ sans modifier l’entraînement.

Le papier utilise un pas fixe pour CIFAR-10 et un pas décroissant pour ImageNet. Nous avons opté pour une décroissance en cosinus du pas ϵ , dont la mise à jour est visible dans le pseudo code ci-dessus. Le pas décroît lentement en début de chaîne pour explorer l’espace latent, rapidement en milieu de chaîne, puis se stabilise progressivement vers $\epsilon_{min} = 10^{-6}$, assurant une transition naturelle entre exploration et convergence.

4 Comparaison et Résultats expérimentaux

Nous évaluons notre implémentation WGAN-GP + DDLS en la comparant au GAN standard (Original), à la Hard Truncation et à la Soft Truncation, via les métriques FID, Precision et Recall sur 10 000 images générées depuis le dataset MNIST.

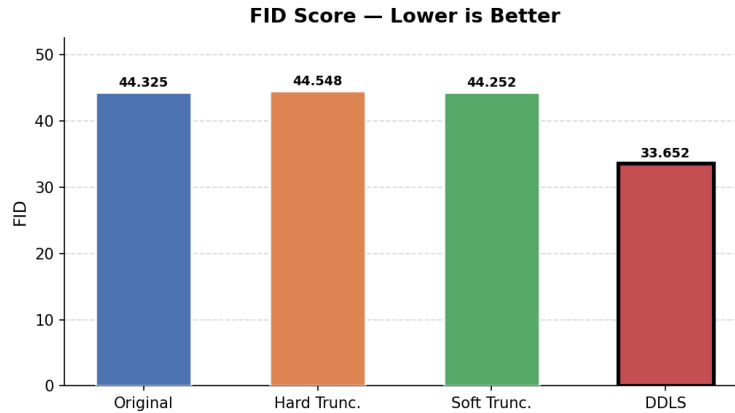


Figure 1: FID Score

FID : DDLS obtient un FID de 33.652, soit une amélioration significative de 24% par rapport au GAN standard (44.325). La Hard Truncation (44.548) et la Soft Truncation (44.252) n’apportent quant à elles aucune amélioration notable par rapport au GAN standard, leurs scores étant quasi-identiques.

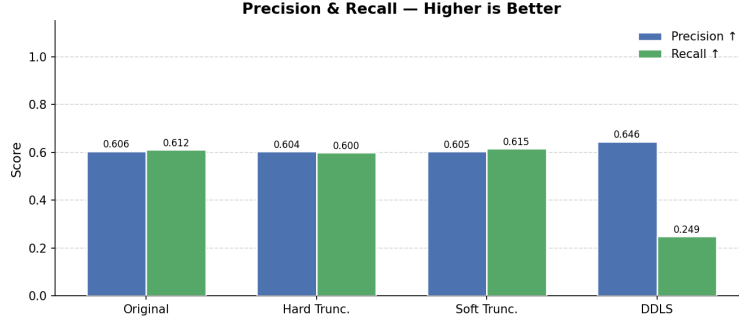


Figure 2: Precision & Recall

Precision : DLSS avec WGAN-GP obtient une meilleure précision avec 0.646, contre 0.606 pour le GAN standard, 0.604 pour la Hard Truncation et 0.605 pour la Soft Truncation. DLSS avec WGAN-GP génère donc des images de meilleure qualité individuelle.

Recall : On observe pour ce que nous avons implémenté un recall de 0.249 contre 0.612 pour le GAN standard, 0.600 pour la Hard Truncation et 0.615 pour la Soft Truncation. Cette chute importante suggère que DLSS avec WGAN-GP favorise la qualité des images au détriment de la diversité.

Synthèse : DLSS avec WGAN-GP améliore significativement le FID et la Precision, mais au prix d’une forte réduction du Recall.

5 Limites de l’implémentation globale

Plusieurs limites sont à noter dans notre implémentation, qu’elles soient liées à l’architecture, à la compatibilité entre les deux méthodes, ou aux contraintes pratiques du projet.

Le DDLS est formulé pour un GAN standard dont le discriminateur produit une probabilité via sigmoid, interprétable comme une log-densité. Le critic WGAN-GP est lui non borné et n’approxime pas la même quantité. L’application d’un sigmoid est une approximation qui permet de combiner les deux méthodes, mais elle n’est pas théoriquement garantie. Cette mauvaise calibration de $d(G(z))$ dans la fonction d’énergie concentre l’échantillonnage sur certaines zones de l’espace latent, réduisant la diversité des images générées.

Le Théorème 1 du papier DDLS garantit $p_d^* = p_d$ uniquement lorsque le discriminateur est optimal. En pratique, le discriminateur n’atteint jamais l’optimalité, ce qui biaise l’échantillonnage DDLS vers les modes que le discriminateur juge les plus réalistes, au détriment des autres modes. Ces deux derniers points expliquent probablement la chute significative du Recall (0.249) observée dans nos résultats, contre 0.612 pour le GAN standard.